

SQL avanzado

Clase 05

IIC2413 - Sección 3

Prof. Cristian Riveros

Outline

Agregación

Funciones y filtros

Orden y cantidad

Outline

Agregación

Funciones y filtros

Orden y cantidad

Agregación en SQL

```
SELECT AGG(<atributo>)  
FROM <lista-relaciones>  
WHERE <condicion>
```

} Consulta de
agregación

Operadores más usados para AGG

- COUNT: **cuenta** la cantidad de valores en <atributo>
- SUM: **suma** los valores en <atributo>
- MAX: **máximo** de los valores en <atributo>
- MIN: **mínimo** de los valores en <atributo>
- AVG: **promedio** de los valores en <atributo>

Importante: una consulta de agregación entrega un **único valor**.

Agregación en SQL: recordatorio ejemplo

Planets

name	dist	radio	grav	years	temp	ring
Mercurio	0.39	0.38	2.8	0.241	440	FALSE
Venus	0.72	0.95	8.9	0.615	730	FALSE
Tierra	1.00	1.00	9.8	1.000	288	FALSE
Marte	1.52	0.53	3.7	1.880	186	FALSE
Júpiter	5.20	10.97	22.9	11.862	152	TRUE
Saturno	9.54	9.14	9.1	29.447	134	TRUE
Urano	19.19	3.98	7.8	84.017	76	TRUE
Neptuno	30.07	3.86	11.0	164.791	53	TRUE

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Triton	Neptuno	Lassell	1845

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

(ejemplo de Aidan Hogan – UChile)

Agregación en SQL: COUNT

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo con COUNT

```
SELECT COUNT(planet) AS Nland  
FROM Landings
```



planet	
Mercurio	}
Venus	
Venus	
Marte	
Marte	
Marte	
Júpiter	

COUNT

Agregación en SQL: COUNT

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo con COUNT

```
SELECT COUNT(planet) AS Nland  
FROM Landings
```



Nland
7

¿cómo contamos los planetas distintos?

Agregación en SQL: COUNT (distintos)

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo con COUNT DISTINCT

```
SELECT DISTINCT COUNT(planet) AS Nland  
FROM Landings
```



Nland
7



Agregación en SQL: COUNT (distintos)

Landings			
spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo con COUNT DISTINCT

```
SELECT COUNT(DISTINCT planet) AS Nland  
FROM Landings
```



Nland
4



AGG(DISTICT <atributo>) nos permite agregar **sin duplicados**

Agregación en SQL: COUNT (tuplas)

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Triton	Neptuno	Lassell	1845

Ejemplo con COUNT(*)

```
SELECT COUNT(*) AS Ntuples  
FROM Satellites
```



Ntuples
7

COUNT(*) nos permite contar **cantidad de tuplas**

Agregación en SQL: COUNT (valores NULL)

Satellites

name	planet	discover	dyear
Luna	Tierra	--	--
Ganímedes	Júpiter	Galileo	1610
Calisto	Júpiter	Galileo	1610
Europa	Júpiter	Galileo	1610
Ío	Júpiter	Galileo	1610
Titán	Saturno	Huygens	1655
Triton	Neptuno	Lassell	1845

Ejemplo con NULL

```
SELECT COUNT(discover) AS Ndisc  
FROM Satellites
```



Ndisc
6

COUNT **no cuenta** los valores NULL

Agregación en SQL: MIN

Landings			
spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo con MIN

```
SELECT MIN(year) AS minmarte  
FROM Landings  
WHERE planet = 'Marte'
```



minmarte
1971

¿es necesario usar DISTINCT en este caso?

Agregación en SQL: mas atributos

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo con más atributos

```
SELECT MIN(year) AS min, MAX(year) AS max  
FROM Landings  
WHERE planet = 'Marte'
```

⇒

min	max
1971	2003

Agregación en SQL: mas atributos

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo con más atributos (2)

```
SELECT planet, MIN(year) AS min  
FROM Landings  
WHERE planet = 'Marte'
```



planet	min
Marte	1971

} SQLite

planet	min
ERROR	ERROR

} PostgreSQL

En general, el resultado debería ser ERROR.

Agregación en SQL: mas atributos

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo con más atributos (3)

```
SELECT planet, MIN(year) AS min  
FROM Landings
```



ERROR	ERROR
-------	-------

} SQLite

ERROR	ERROR
-------	-------

} PostgreSQL

¿cómo podemos listar cada planeta con el año de su primer aterrizaje?

Agregación en SQL: mas atributos

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Primer intento

```
SELECT planet, MIN(year) AS min
FROM Landings WHERE planet = 'Marte'
UNION
SELECT planet, MIN(year) AS min
FROM Landings WHERE planet = 'Venus'
UNION ...
```



planet	min
Marte	1971
Venus	1966
Mercurio	2015
Júpiter	2003

¿qué pasa si tenemos **un nuevo planeta**?

Agregación en SQL: GROUP BY

```
SELECT <atr1>, AGG(<atr2>)  
FROM <lista-relaciones>  
WHERE <condicion>  
GROUP BY <atr1>
```

≡

```
SELECT <atr1>, AGG(<atr2>)  
FROM <lista-relaciones>  
WHERE <condicion>  
      AND <atr1> = 'v1'  
UNION  
SELECT <atr1>, AGG(<atr2>)  
FROM <lista-relaciones>  
WHERE <condicion>  
      AND <atr1> = 'v2'  
UNION  
...  
UNION  
SELECT <atr1>, AGG(<atr2>)  
FROM <lista-relaciones>  
WHERE <condicion>  
      AND <atr1> = 'vk'
```

donde $\{v1, \dots vk\}$

son todos los valores en <atr1>

Agregación en SQL: GROUP BY

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo de GROUP BY / MIN

```
SELECT planet, MIN(year) AS min  
FROM Landings  
GROUP BY planet
```



planet	min
Marte	1971
Venus	1966
Mercurio	2015
Júpiter	2003

Agregación en SQL: GROUP BY (múltiples)

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	EEUU	2003
Galileo	Júpiter	EEUU	2003

Ejemplo de GROUP BY / COUNT (múltiples)

```
SELECT planet, ctry,  
       MIN(*) AS min  
FROM Landings  
GROUP BY planet, ctry
```



planet	ctry	min
Mercurio	EEUU	2015
Venus	URSS	1966
Venus	EEUU	1978
Marte	URSS	1971
Marte	EEUU	1976
Júpiter	EEUU	2003

Agregación en SQL: GROUP BY / COUNT

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo de GROUP BY / COUNT

```
SELECT planet, COUNT(*) AS count  
FROM Landings  
GROUP BY planet
```



planet	count
Marte	3
Venus	2
Mercurio	1
Júpiter	1

¿cómo podemos filtrar con respecto a un **grupo**?

¿cómo podemos filtrar con respecto a un grupo?

```
SELECT <atr1>, AGG(<atr2>)  
FROM <lista-relaciones>  
WHERE <condicion>  
GROUP BY <atr1>
```

Orden de ejecución de consulta SQL

1. FROM
2. WHERE
3. GROUP BY
4. SELECT

¿cómo podemos **filtrar los grupos** después del GROUP BY?

Agregación en SQL: HAVING

```
SELECT <atr1>, AGG(<atr2>)
FROM <lista-relaciones>
WHERE <condicion-tupla>
GROUP BY <atr1>
HAVING <condicion-grupo>
```

≡

```
SELECT <atr1>, AGG(<atr2>)
FROM <lista-relaciones>
WHERE <condicion-tupla>
      AND <atr1> = 'v1'
      "AND <condicion-grupo>"
UNION
...
UNION
SELECT <atr1>, AGG(<atr2>)
FROM <lista-relaciones>
WHERE <condicion-tupla>
      AND <atr1> = 'vk'
      "AND <condicion-grupo>"
```

donde $\{v1, \dots vk\}$
son todos los valores en <atr1>

La condición HAVING es a **nivel de grupo** y NO de tupla

Agregación en SQL: HAVING

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo de HAVING

```
SELECT planet, COUNT(*) AS count
FROM Landings
GROUP BY planet
HAVING planet = 'Marte'
```



planet	count
Marte	3

Condición de grupo (sin agregación) **solo puede usar** atributos en GROUP BY

Agregación en SQL: HAVING

Ejemplo de HAVING

```
SELECT planet, COUNT(*) AS count  
FROM Landings  
GROUP BY planet  
HAVING planet = 'Marte'
```



planet	count
Marte	3

¿és posible mover el filtro de HAVING al WHERE?



```
SELECT planet, COUNT(*) AS count  
FROM Landings  
WHERE planet = 'Marte'  
GROUP BY planet
```



planet	count
Marte	3

- **Solo cuando** el atributo aparece en el GROUP BY.
- Es **buena práctica** mover este tipo de filtro al WHERE.

Agregación en SQL: HAVING con agregación

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo de HAVING con agregación

```
SELECT planet, COUNT(*) AS count
FROM Landings
GROUP BY planet
HAVING MAX(year) <= 2000
```



planet	count
Venus	2

Agregación en SQL: HAVING

Tipos de condición de grupo en HAVING

1. Con atributo en GROUP BY.
 - Esta condición (filtro) la podemos llevar al WHERE.
2. Con agregación.

Condición de grupo **con agregación** es el propósito principal de HAVING

Outline

Agregación

Funciones y filtros

Orden y cantidad

Funciones en SQL

Funciones aritméticas

- $+$ $-$ $\backslash$ $*$ $\%$
- `ABS(a)`
- `CEIL(a)`
- `FLOOR(a)`
- `ROUND(a)`
- `SQRT(a)`
- `EXP(a,b)`
- ...

Funciones strings

- `LOWER(a)`
- `UPPER(a)`
- `TRIM(a)`
- `SUBSTRING(a,b)`
- `STARTSWITH(a)`
- ...

Muchas funciones tienen variaciones dependiendo del motor

Funciones en SQL

Planets

name	dist	radio	grav	years	temp	ring
Mercurio	0.39	0.38	2.8	0.241	440	FALSE
Venus	0.72	0.95	8.9	0.615	730	FALSE
Tierra	1.00	1.00	9.8	1.000	288	FALSE
Marte	1.52	0.53	3.7	1.880	186	FALSE
Júpiter	5.20	10.97	22.9	11.862	152	TRUE
Saturno	9.54	9.14	9.1	29.447	134	TRUE
Urano	19.19	3.98	7.8	84.017	76	TRUE
Neptuno	30.07	3.86	11.0	164.791	53	TRUE

Ejemplo de uso de función

```
SELECT name,  
       ABS(dist-1.0) AS earthdist  
FROM Planets
```



name	earthdist
Mercurio	0.61
Venus	0.28
Tierra	0.0
Marte	0.52
Júpiter	4.2
Saturno	8.54
Urano	18.19
Neptuno	29.07

Funciones se pueden usar en SELECT, FROM, o HAVING.

Filtros avanzados en SQL

■ <atributo> IN (<lista-valores>)

Ejemplo: aterrizajes de EEUU o ESA

```
SELECT spaceship  
FROM Landings  
WHERE ctry IN ('EEUU', 'ESA')
```

Filtros avanzados en SQL

- `<atributo> IN (<lista-valores>)`
- `<atributo> BETWEEN <lower> AND <upper>`

Ejemplo: aterrizajes entre 2000 y 2025

```
SELECT spaceship  
FROM Landings  
WHERE year BETWEEN 2000 AND 2025
```

Filtros avanzados en SQL

- `<atributo> IN (<lista-valores>)`
- `<atributo> BETWEEN <lower> AND <upper>`
- `<atributo> LIKE <patron>`

Ejemplo: aterrizajes de naves que empiezan con M

```
SELECT spaceship  
FROM Landings  
WHERE spaceship LIKE 'M%'
```


Filtros avanzados en SQL

- `<atributo> IN (<lista-valores>)`
- `<atributo> BETWEEN <lower> AND <upper>`
- `<atributo> LIKE <patron>`
- `<atributo> NOT LIKE <patron>`

Ejemplo: aterrizajes de naves que **no** contienen m

```
SELECT spaceship
FROM Landings
WHERE spaceship NOT LIKE '%m%'
```

Filtros avanzados en SQL

- `<atributo> IN (<lista-valores>)`
- `<atributo> BETWEEN <lower> AND <upper>`
- `<atributo> LIKE <patron>`
- `<atributo> NOT LIKE <patron>`

`<atributo> LIKE 'pref%'` ≡ **Empieza** con palabra "pref"

`<atributo> LIKE '%suf'` ≡ **Termina** con palabra "suf"

`<atributo> LIKE '%sub%'` ≡ **Contiene** la palabra "sub"

Outline

Agregación

Funciones y filtros

Orden y cantidad

Orden y cantidad de resultados en SQL

```
SELECT <lista-atributos>
FROM <lista-relaciones>
WHERE <condicion>
ORDER BY <atrib1> [orden], ..., <atribk> [orden]
LIMIT N
```

} Orden y Cantidad

[orden] = ASC (ascendente)

[orden] = DESC (descendente)

ORDER BY y LIMIT son solo usados para la **visualización** de los resultados

Orden y cantidad de resultados en SQL

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo de ORDER BY

```
SELECT planet, year  
FROM Landings  
ORDER BY year ASC, planet ASC
```



planet	year
Venus	1966
Marte	1971
Marte	1976
Venus	1978
Júpiter	2003
Marte	2003
Mercurio	2015

Orden y cantidad de resultados en SQL

Landings

spaceship	planet	ctry	year
Messenger	Mercurio	EEUU	2015
Venera 3	Venus	URSS	1966
Pioneer	Venus	EEUU	1978
Mars 2	Marte	URSS	1971
Viking 1	Marte	EEUU	1976
Beagle 2	Marte	ESA	2003
Galileo	Júpiter	EEUU	2003

Ejemplo de ORDER BY + LIMIT

```
SELECT planet, year
FROM Landings
ORDER BY year ASC, planet ASC
LIMIT 3
```



planet	year
Venus	1966
Marte	1971
Marte	1976

¿qué tan estándar es el estándar de SQL?

SQLite / Postgres / MySQL

```
SELECT planet, year  
FROM Landings  
ORDER BY year ASC, planet ASC  
LIMIT 3
```

SQL Server / MS Access

```
SELECT TOP 3 planet, year  
FROM Landings  
ORDER BY year ASC, planet ASC
```

Oracle

```
SELECT planet, year  
FROM Landings  
ORDER BY year ASC, planet ASC  
OFFSET 0 ROWS FETCH NEXT 3 ROWS ONLY;
```

¿qué tan estándar es el estándar de SQL?

Dialecto SQL = Modificación al lenguaje SQL
no incluida en el estándar de SQL

¿por qué un motor no seguiría el estándar de SQL?

1. Innovación / Compatibilidad versión anterior
2. Por eficiencia
3. Atrapar al usuario

Recomendación: En lo posible, no utilizar dialectos de SQL.